From Road Safety to Smarter AI

Author: Rula Awad
Let contributor: Rula Awad
Acknowledgements: Yasmin Fatihy, Mona Jaber,
Project: Techkies Workshops

High-level Table of Contents

From Road Safety to Smarter AI: A diffusion-first tutorial for imbalanced sensor data + LLM RAG

1. ***Workshop Agenda***
2. ***Overview and Objectives***
3. ***What You Will Learn***
4. ***ITS + Road Safety Context***
5. ***Problem Framing: Rare Events + Cost-Sensitive Classification***
6. ***Dataset: APS Failure at Scania Trucks***
7. ***High-Level Pipeline and Notebooks Map***
    - *7.1 Technical Framework Diagram*
    - *7.2 Imbalance Handling Taxonomy*

8. ***Notebook 01: Preprocessing + PCA***
    - *8.1 Data ingestion*
    - *8.2 Imputation and scaling*
    - *8.3 PCA rationale and outputs*

9. ***Notebook 02: Clustering Failure Samples (Diagnostic)***
    - *9.1 Why clustering*
    - *9.2 Interpreting cluster structure (multi-modal failures)*

10. ***Notebook 03: Diffusion Augmentation for Tabular PCA Vectors***
    - *10.1 Diffusion model intuition (forward noise / reverse denoise)*
    - *10.2 Minority augmentation options (oversampling, SMOTE, diffusion)*
    - *10.3 Why diffusion (training stability, multi-modality coverage)*
    - *10.4 Sampling synthetic failures + saved artifacts*
    - *10.5 SMOTE explainer*
    - *10.6 Baselines*
    - *10.7 Quality checks (KS-test)*

11. ***Notebook 04: Cost-Sensitive Evaluation***

- *11.1 Baseline vs diffusion-augmented training comparison*
- *11.2 Threshold moving via validation cost minimization (+ ROC visuals placeholder)*

© 2026 Rula Awad. All rights reserved.

- *11.3 Metrics definitions (Precision/Recall/F1)*
- *11.4 IR metric + KS-test heuristics (**placeholder)***
- *11.5 Results summary **(placeholder)***

12. Notebook 0X: LLMs + RAG (Grounded Answers) (placeholders)

- *12.1 LLM + Transformer primer (placeholder diagrams from slides)*
- *12.2 RAG in one paragraph*
- *12.3 RAG pipeline + architecture (placeholder diagrams from slides)*
- *12.4 What you control (URL, chunking, embeddings, top-k, prompt rules)*
- *12.5 agents*

13. Key Takeaways (placeholder)

14. Appendix: Troubleshooting Checklist

15. References

A diffusion-first tutorial for imbalanced sensor data + LLM RAG

The objective of this tutorial is to walk through a real-world applied AI problem end to end. Rather than focusing on a single model or metric, the tutorial demonstrates how to structure a complex applied AI problem, make principled design choices at each stage, and evaluate the resulting system in a way that reflects real operational priorities.

This guide is meant to be read alongside the notebooks in this project. It explains the problem setting, the reasoning behind each step of the pipeline, and the core theory needed to understand diffusion-based minority-class data augmentation and Retrieval-Augmented Generation (RAG) for grounding and improving large language model outputs.

In the predictive maintenance portion of the tutorial, we work in a realistic transportation context with highly imbalanced sensor data. We frame failure detection as a cost-sensitive classification problem, where missing rare but critical failures is far more expensive than raising false alarms. To address the challenges posed by severe class imbalance (a common problem in AI classification problems), we replace earlier GAN-based approaches with a DDPM-style diffusion model, which offers more stable training behavior and better coverage of complex, potentially multi-modal minority-class distributions.

Importantly, the tutorial does not assume that more complex models are automatically better. Instead, it emphasizes controlled comparisons: training the same downstream classifier with and without diffusion-based augmentation, selecting decision thresholds using an explicit cost function, and evaluating performance on held-out data. This evaluation mindset ensures

© 2026 Rula Awad. All rights reserved.

that improvements are measured in terms of decision quality and real-world cost, not just headline accuracy.

A separate notebook demonstrates Retrieval-Augmented Generation (RAG) as a complementary capability, showing how large language models can be grounded in external documents to produce more reliable, evidence-based answers. While independent of the predictive maintenance pipeline, this component illustrates how modern AI systems can combine learned models with retrieval to improve trustworthiness and usability.

Together, the notebooks and this guide present a cohesive example of how to design, augment, and evaluate AI systems for high-stakes, real-world applications in transport systems.

The workshop "Agenda"

Workshop Agenda

- **Welcome + objectives** (what you'll build, what *success* means)
- **ITS and road safety context** (why rare events and synthetic data matter)
- **Case study I: Predictive maintenance with imbalanced APS sensor data (diffusion-first)**
    - Notebook 01: **Preprocessing + PCA**
    - Notebook 02: **Failure clustering** (diagnostic for multi-modal failures)
    - Notebook 03: **Diffusion-based minority augmentation**
    - Notebook 04: **Cost-sensitive evaluation and threshold moving**
- **Large Language Models (LLMs)** (a primer, why they matter in transport)
- **Retrieval-Augmented Generation (RAG)** (how grounding reduces hallucinations)
- **Case study II: LLaMA + Agentic RAG for context-aware insights in transport** (Notebook 0X)
- **Wrap-up, key takeaways, next steps**

What you will learn

By the end, you should be able to:

- ☐ Frame a real world applied AI problem end to end. We have chosen a predictive maintenance problem as a cost-sensitive classification task.
- ☐ Prepare a dataset, for us this is the APS Failure dataset: handle missingness, scaling, and PCA dimensionality reduction.
- ☐ Diagnose minority-class structure with clustering (are there multiple failure modes?).

© 2026 Rula Awad. All rights reserved.

- ☐ Train a simple DDPM-style diffusion model on tabular PCA vectors and sample synthetic failures.
- ☐ Evaluate models using cost, precision/recall, and an explicit threshold selection step.
- ☐ **Bonus 0X:** Build a basic real life RAG pipeline with our dataset (URL -> chunks -> embeddings -> retrieval -> grounded LLM answer).
- ☐ Have a portfolio ready assignment for AI and data science job applications and interviews.

1. ITS & road safety context: why generative data matters

Intelligent Transportation Systems (ITS) sit at the intersection of vehicles, infrastructure, road users, and decision-making software. A major driver of modern ITS is the rise of ADAS and self-driving car systems, where machine learning supports perception (detecting vehicles, pedestrians, lanes), prediction, and decision-making under real-world complexity: changing weather and lighting, unpredictable road-user behavior, shifting traffic patterns, sensor noise, and operational constraints.

A recurring challenge is that the most safety-critical and operationally important events are often rare ie. statistical long tail events: near-misses, unusual pedestrian maneuvers, rare object configurations, and low-frequency component failures. These are the most important scenarios we want our models to handle correctly. Collecting enough real-world examples can be slow, expensive, and sometimes difficult due to privacy, safety, or regulatory constraints.

This is where **generative data** becomes practically valuable in ITS. It can help teams:

- **Expand coverage** of rare but important scenarios (edge cases),
- **Test systems** in controlled variations (lighting, weather, density, occlusions),
- **Reduce dependence** on sensitive real-world data,
- **Accelerate iteration** when real data collection is the bottleneck.

Examples of generative ai applied to ITS

Synthetic data for autonomous vehicle training: improving object detection coverage for the vehicle object class.

[FIGURE PLACEHOLDER: Synthetic Data for Autonomous Vehicle Training - vehicle object class]

Synthetic data for autonomous vehicle training: improving detection and robustness for the pedestrian object class (a key safety-critical category).

© 2026 Rula Awad. All rights reserved.

[FIGURE PLACEHOLDER: Synthetic Data for Autonomous Vehicle Training - pedestrian object class]

Traffic pattern generation for smart cities: generating realistic traffic flows/patterns for planning, simulation, and operational stress-testing.

[FIGURE PLACEHOLDER: Traffic Pattern Generation for Smart Cities]

Pedestrian behaviour modelling: supporting safety analysis and interaction modelling in complex urban environments.

[FIGURE PLACEHOLDER: Pedestrian Behaviour Modelling]

Road safety & fleet management framing: positioning AI systems within real operational workflows (monitoring, compliance, safety KPIs, and interventions).

[FIGURE PLACEHOLDER: Road Safety & Truck fleet management]

In this workshop, we apply the same motivation to predictive maintenance: failures are rare, high-impact, and underrepresented in the training set, so we explore diffusion-based minority-class augmentation as one way to improve decision quality under severe class imbalance.

Industry context: maintenance planning as a safety and uptime lever Scania Flexible Maintenance Plans

In fleet operations, maintenance planning is not only about cost control, it is also a lever for road safety, uptime, and operational reliability. Even small improvements in how failures are detected and handled can translate into fewer immobilizations, better asset availability, and reduced safety risk.

[PLACEHOLDER: Scania Maintenance Flexible Plan slide image]

Because maintenance decisions have asymmetric consequences, missing a true failure is far more costly than a false alarm, this tutorial evaluates models using an explicit cost function and threshold selection rather than relying on accuracy alone.

1.2 Framing the problem: high-stakes predictions with rare events

© 2026 Rula Awad. All rights reserved.

Modern transportation systems collect massive sensor streams (vehicles, fleets, infrastructure). The most important events are often the hardest to model because of their low frequency, typically appearing as rare failures and edge cases. In predictive maintenance, a model that misses *true failures* can cause downtime, safety risk, and major cost.

Why class imbalance is central

In our APS (Air Pressure System) dataset, failures comprise a small minority of samples. If we train and evaluate naively, a model can achieve high overall accuracy by predicting "no failure" most of the time. That is not a useful model. We need methods and metrics that focus on understanding and discerning the minority class and the real cost of mistakes.

Cost-sensitive framing

Instead of treating all errors equally, we use a cost function:

Cost = CFP * FP + CFN * FN

Where:

- FP is a false positive - the cost of this is an unnecessary maintenance action
- FN is a false negative - missing a true failure and thus costing the business more in missed business opportunity on top of the cost of an immobilised vehicle

In this project we use the APS challenge costs: CFP = 10, CFN = 500. This makes the goal explicit: reduce missed failures even if it slightly increases false alarms.

2. Dataset: APS Failure at Scania Trucks

The APS generates pressurised air for functions such as braking, suspension, seats, and gearbox. A fault can immobilize a vehicle. Sensors are collected at different intervals and the dataset includes 170 anonymized features plus a label.

Practical data issues that must be handled

- Missing values are encoded as the string 'na'
- The raw CSVs include 20 metadata rows at the top; the actual header starts at row 21.
- Severe class imbalance: normal samples greatly outnumber failures.
- High dimensionality: many correlated sensor features.

What we ultimately want to conclude

We are trying to answer a specific question:

© 2026 Rula Awad. All rights reserved.

Does diffusion-based minority augmentation improve our ability to detect APS failures and reduce total cost, without destroying precision or creating unrealistic samples?

To answer that, we need:

- a clean baseline dataset,
- a clear evaluation protocol, and
- a way to compare baseline vs augmented training in a fair way.

3. High-level pipeline

To include: The analytical framework diagram

We have 4 notebooks for the first case study of this tutorial. Each notebook produces artifacts used by the next. In the bonus notebook we build context aware system for transport. we cover context Recommended run order:

Preprocess + PCA -> Failure clustering -> Diffusion augmentation -> Cost-sensitive evaluation

There is also a separate RAG demo notebook that is not connected to the APS pipeline.

Notebooks	Purpose	Key inputs	Key outputs	Time Budget tp go through each notebook
Preprocess + PCA	Clean data; handle missingness; scale the data; export PCA features	Raw APS train/test CSVs	Imputed CSVs + PCA CSVs	20 - 35 mins
Failure clustering	Diagnose whether failures form sub-types (modes)	PCA train/test CSVs	Cluster labels + silhouette metrics	15 - 30 mins
Diffusion augmentation	Train diffusion model on failure PCA vectors and sample synthetic failures	PCA train CSV	Synthetic failures + augmented train CSV + checkpoints	20 - 30 mins (for workshop purposes)
Cost-sensitive evaluation	Train baseline vs augmented classifier; pick threshold by cost; evaluate on test	PCA train/test + augmented train (optional)	Cost + metrics + threshold plots	20 - 30 mins
Bonus: Agentic RAG Search	Separate demo: grounded Q&A via RAG	A URL	Retrieval + LLM answers (+ optional UI)	30-40 mins

3.1 The Technical Framework + Diagram

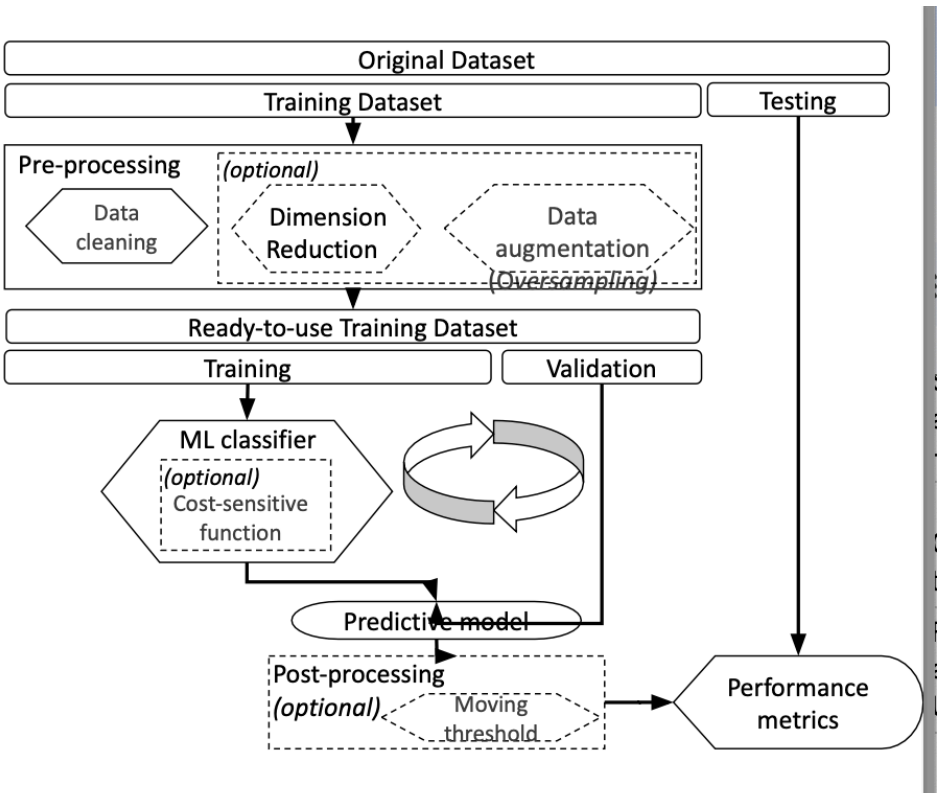
© 2026 Rula Awad. All rights reserved.

This tutorial follows a structured technical framework to make sure our results are comparable, reproducible, and aligned with real operational goals. The key idea is that in real transportation settings (including predictive maintenance), a model is only useful if it supports better decisions, not just better headline metrics like accuracy.

At a high level, the workflow is:

1. **Start with an original dataset split** (training vs. testing), so we can evaluate the final system on held-out data.
2. **Pre-process the training data** (handle missing values, scaling, cleanup). Sensor datasets are messy and most ML models cannot handle strings or unscaled feature ranges.
3. **Reduce dimensionality (PCA)** to create a compact continuous feature space that is easier to cluster for diagnostics, and practical for training a diffusion model on tabular data.
4. **Augment minority-class samples** to address class imbalance.We use diffusion to generate synthetic failure samples using a DDPM-style diffusion model trained on the failure class in PCA space. We then create an augmented training set.
5. **Train the downstream classifier** under a controlled comparison:
    - Baseline model trained on the original training set
    - Augmented model trained on the diffusion-augmented training set
      (same downstream classifier and evaluation protocol, so the comparison is uniform and fair)
6. **Choose a decision threshold using validation cost minimization**, rather than assuming a default threshold e.g., 0.5. In cost-sensitive, imbalanced settings, threshold selection is critical.
7. **Evaluate once on the test set** using the selected threshold and report:
    - ➔ total cost
    - ➔ false positives / false negatives
    - ➔ precision / recall (and any additional diagnostic plots as needed)

© 2026 Rula Awad. All rights reserved.



3.2 Imbalance handling taxonomy

Goal: make the model 'see and understand' more of the minority-class signal during training by changing the dataset's class distribution or the available minority examples.

We can use:

- **Oversampling to increase minority samples**
  Naive oversampling: duplicate minority rows (fast, but can overfit)
  - *SMOTE:* generate synthetic minority samples via interpolation between minority neighbours.
  - *Generative augmentation:* learn the minority distribution and sample new failures e.g. diffusion- or GANs-based as in this and the previous workshop.
- **Undersampling to reduce majority samples**
  - Randomly remove majority samples to balance classes
  - Can help speed and balance training, but risks throwing away useful information.

When data-driven methods help most: When the minority class is underrepresented but still has a learnable structure ie, you can generate or sample additional minority examples that improve coverage of the minority space.

© 2026 Rula Awad. All rights reserved.

Our method uses a diffusion-based minority augmentation (training a diffusion model on failure samples in PCA space, then sampling synthetic failures).

3.2.2 Cost-sensitive approaches to change the learning objective or decision rule

Goal: keep the dataset as-is, but change the algorithm and/or decision policy so minority mistakes are treated as more serious.

This includes:

- **Algorithm-level cost sensitivity**
  - Apply class weights or cost-weighted losses (eg, weighted cross-entropy)
  - Use models designed to optimise imbalanced objectives (varies by algorithm)

- **Post-processing / decision-level cost sensitivity**
  - **Threshold moving:** instead of using a default 0.5 cutoff, choose a threshold that reflects operational tradeoffs
  - This is often the most direct way to align model behavior with real-world because an overlooked failure is more costly than a false positive

When cost-sensitive methods help most: When you can't easily change the data distribution but you *can* change how the model is trained or how decisions are made from its probabilities.

We use cost-sensitive evaluation and threshold selection: we scan thresholds on a validation set and pick the threshold that minimizes total cost, then evaluate once on test.

3.2.3 Hybrid approaches: data-driven and cost-sensitive

Goal: get the best of both worlds by improving minority coverage and aligning training/evaluation with real-world costs.

Hybrid approaches include:

- **Resampling and cost-sensitive training**
  - Oversample minority samples *and* apply class weights.

- **Resampling and threshold moving**
  - Augment data to improve recall, then tune the threshold to control false alarms.

- **Ensemble / hybrid frameworks**
  - Combine multiple models or strategies eg, ensembles that balance different error profiles

Why hybrid approaches: Imbalanced problems usually fail for *two reasons at once*: too few minority examples and the wrong objective/threshold for deployment. Hybrids address both failure modes

© 2026 Rula Awad. All rights reserved.

We use a hybrid method because we combine diffusion augmentation (the data driven) and a cost sensitive threshold moving (cost sensitive decision rule)

That combination is what ultimately determines whether the system improves operational outcomes (lower cost), not just whether it generates plausible samples.

Some heuristics

- If you only change the data you may improve recall but risk more false alarms
- If you only change the decision policy (cost-sensitive) you may control alarms but still be limited by weak minority learning.
- In high-stakes ITS problems, hybrid approaches are often optimal: *augment minority coverage* + *evaluate/decide using the true operational objective*.

4. Preprocessing + dimensionality reduction (PCA)

The Goal of this notebook is to turn raw APS CSVs into clean numeric matrices and PCA features.

This notebook does four important things:

1. label encoding (neg/pos -> 0/1)
2. missing-value handling
3. scaling + imputation
4. PCA export

4.1 Reading the raw APS CSVs

The APS CSVs begin with a metadata block. Use skiprows=20. Also, missing values are stored as the string "na". You must convert them to real NaN values before imputation.

- `df = pd.read_csv(..., skiprows=20)`

Then either:

- `df.replace("na", np.nan)`

or pass `na_values=["na"]` to read_csv.

4.2 Scaling and imputation

PCA and most classifiers cannot accept missing values. We impute missing values (mean imputation in this baseline).

We also scale features so that no single sensor dominates due to units or magnitude. Scaling is particularly important before PCA and distance-based methods to ensure calibration between the

© 2026 Rula Awad. All rights reserved.

4.3 PCA: why and how many components?

PCA is a statistical technique that reduces the dimensionality of a dataset while preserving as much of its variability as possible. It works by transforming the original sensor features into a smaller set of new, uncorrelated variables called *principal components*, which are ordered by how much variance they explain in the data with the first component capturing the greatest variance. In this tutorial we use 11 components. That is a practical compromise that gives us a set that is:

- small enough for fast clustering and diffusion training
- large enough to retain much of the useful variance as possible

Output files contain pc_0 ... pc_10 plus the class label.

5. Notebook 02: clustering failure samples

[clustering output from notebooks]
[add section on conditionality?]

The Goal: to understand whether *failure* is a single homogeneous group or whether it consists of multiple sub-types.

We use clustering, an unsupervised learning technique that identifies natural groupings in data without predefined labels. By grouping samples that are more similar to each other than to the rest of the data, clustering helps reveal hidden structure, patterns, or sub-populations. In our setting, it serves as a diagnostic tool to assess whether the data exhibits meaningful internal structure before committing to more complex modeling choices.

Why this is useful:

If failures are multi-modal, a generative model must cover multiple regions of the feature space rather than collapsing to a single mode. Identified clusters can be used for conditional generation (optional) or for validation, such as checking whether generated samples are sufficiently diverse.

6. Notebook 03: diffusion augmentation for tabular PCA vectors

This notebook is the core generative step. We train a DDPM-style diffusion model on failure-only PCA vectors and sample synthetic failures to balance the training set.

What is a diffusion model?

© 2026 Rula Awad. All rights reserved.

A diffusion model is a generative model that learns to produce new samples by reversing a gradual noising process.

- **Forward process (noising):** start from a real data point x_0 and repeatedly add small Gaussian noise over many steps, producing $x_1, x_2, \ldots, x_T$. After enough steps, x_T becomes (approximately) pure Gaussian noise.

- **Reverse process (denoising):** train a neural network to undo those noise steps: given a noisy sample x_t, predict the noise (or equivalently the denoised signal) so you can step back to $x_{t - 1}$. After iterating from time $T \rightarrow 0$, you end up with a synthetic sample that looks like it came from the data distribution.

Intuitively, a diffusion model learns how to clean noise into data.

What is minority augmentation?

In our particular classification problem, we do not have enough data points in our failure class so we need to create more synthetic data to reduce imbalance. The goal is usually to improve performance on the rare class by giving the downstream model more diverse minority samples.

We can do this in multiple ways

- Simple oversampling (duplicate rows) which can overfit
- SMOTE-style interpolation which can miss complex, multi-modal patterns
- Generative modeling (GANs (which we used in our last workshop), VAEs, diffusion) which creates more realistic diversity

Diffusion-based minority augmentation: train diffusion only on minority samples (eg, failures), then sample many synthetic failures and add them to your training set.

Why diffusion for minority augmentation?

Diffusion models are a strong fit for minority augmentation because they combine training stability with expressive generation.

It is a simple regression problem:

- The model learns to predict the Gaussian noise that was added
- The loss is normally a mean squared error (MSE)

This is often more stable than adversarial training (which we used in our last workshop), because:

- There is no discriminator/adversary, so gradients are not fighting another network.
- You avoid common GAN failure modes like mode collapse

© 2026 Rula Awad. All rights reserved.

Minority classes (like failures) are heterogeneous so you get different failure models, multiple clusters / subtypes, and nonlinearity so they are able to represent complex, multi-modal distributions which is important if the minority is not from one tight cluster.

Why diffusion in tabular settings?

Tabular data has a few challenges:

- High dimensionality (many correlated features),
- Noise, measurement artifacts,
- Mixed feature types (continuous, categorical, counts).

A common practical strategy is to train diffusion in a compressed continuous space, we do this via PCA to reduce dimensionality. We standardise the features (mean 0 std 1) and train diffusion on the PCA coordinates.

Benefits:

- Lower dimension so easier learning, faster training
- PCA can reduce noise and remove redundant correlations.
- The diffusion model operates in a purely continuous space.

Diffusion models are a strong fit for minority augmentation because they combine training stability with expressive generation.

The forward noising process

Define a sequence of noise levels beta_t. Let alpha_t = 1 - beta_t and alpha_bar_t = product(alpha_s).

We can sample a noisy version of a data vector x0 at time t using:

x_t = sqrt(alpha_bar_t) * x_{0} + sqrt(1 - alpha_bar_t) * eps

where eps ~ N(0, I).

The reverse denoising model

We train a neural network eps_theta(x_t, t) to predict eps.

The standard DDPM loss is:

L = E[|| eps - eps_theta(x_t, t) ||^2]

In practice:

- sample a random timestep t
- sample noise eps
- construct x_{t}

© 2026 Rula Awad. All rights reserved.

- minimize MSE between predicted and true noise

6.4 Sampling synthetic failures

To generate new samples, start with pure noise x_{T} ~ N(0, I) and iteratively denoise down to x_{0}.

At the end, we reverse the standardization transform and write synthetic vectors into a CSV.

The tutorial notebook also saves:

- the trained denoiser weights
- the mean/std used to standardize failures

6.5 Smote explainer

SMOTE (Synthetic Minority Over-sampling Technique) is a baseline for handling class imbalance. SMOTE creates *new* minority samples by interpolating between existing minority points in feature space. SMOTE introduces new training points rather than repeating copies (like duplications) so has less overfitting, and reduces minority recall by filling gaps between minority examples.

Intuition: if two minority (failure) samples are “near” each other, then points in-between them might also be plausible minority samples.

Step-by-step [include the maths]:

1. Pick a minority sample
2. Find nearest minority neighbours
3. Choose one neighbour at random
4. Interpolate along the line segment
5. Repeat until the desired balance is reached

[FIGURE PLACEHOLDER: SMOTE diagram showing minority points, neighbour selection, and an interpolated point from slides]

Limitations: why we chose SMOTE as a baseline

Linear interpolation is simple and doesn't always capture complex, multi-modal minority structure, e.g. the many types of failures that can arise. However, we are only using SMOTE is a good baseline to compare against diffusion, because both aim to increase minority coverage but diffusion can learn more complex structure than simple linear interpolation

6.6 Baselines

© 2026 Rula Awad. All rights reserved.

Diffusion augmentation is only worth using if it improves the downstream decision outcome ie lower cost, not just sample generation. To isolate the effect of augmentation, we compare against the following baselines (run all, or the most relevant subset):

A) No augmentation (reference baseline) Train on the original imbalanced training data → choose threshold by validation cost minimization → evaluate once on test.

B) Random oversampling: Duplicate minority (failure) samples to a target ratio → train → same threshold selection + test evaluation. *Risk:* overfitting to repeated examples.

C) SMOTE: Generate synthetic minority samples via SMOTE → train → same threshold selection + test evaluation.

D) Diffusion augmentation (this tutorial's method): Train diffusion on failure samples (in PCA space) → sample synthetic failures → augment training set → train → same threshold selection + test evaluation.

[TABLE PLACEHOLDER: Baseline comparison table]

6.7 [placeholder: Quality Checks (KS tests)]

The two sample Kolmogorov-Smirnov (KS) compares empirical distributions of feature against its generated set. For our PCA created features, we are comparing:

- ➔ Real failure samples: X_{real}
- ➔ Diffusion generated samples: X_{gen}

The KS statistic measures the maximum distance between between their empirical CDFs:

$$D = \max_{x} \left| F_{\text{real}}(x) - F_{\text{gen}}(x) \right|$$

A smaller D means the distribution is closer to the real distribution for that feature. We run this across multiple OCA dimensions to get summary statistics:

- ➔ An average/median KS statistic across components
- ➔ Max KS statistic (worst component)
- ➔ Number or percentage components with large KS differences

[FIGURE PLACEHOLDER: KS test results summary]

One thing to note about this heuristic is its univariate. It checks each feature dimension independently and does not embody feature relationships (correlations, joint structure).

We can use other sanity checks in tandem to measure health of generated data:

- **visual overlap in PCA space**: [figure placeholder: PCA scatter of real vs synthetic failures]
- **Nearest neighbour distance check**: synthetic samples should not all be exact duplicates of a few real points (collapse), and should not be extremely far from the real minority manifold (outliers)

© 2026 Rula Awad. All rights reserved.

- **Cluster coverage check**: are synthetic samples spread across the same clusters as real failures, or do they concentrate in one region?

7. Notebook 04: cost-sensitive evaluation

A generative model is only useful in our case if it improves downstream decisions i.e., the real operational action taken from model outputs.

In this project, the downstream decision is: *should we flag an example as an APS failure and trigger inspection/maintenance, or not?* The cost of mistakes is asymmetric: false positives (unnecessary checks) are cheaper than false negatives (missing real failures).

This notebook compares two training setups:

- **Baseline**: train a classifier on the original training data (no augmentation)
- **Diffusion-augmented**: train the same classifier on training data augmented with synthetic minority-class (failure) samples generated by diffusion

Key idea: we don't just evaluate the classifier at a fixed threshold. We choose the decision threshold by minimizing cost on a validation split, then evaluate once on the test set with that chosen threshold (to avoid test leakage). This aligns evaluation with the real objective:

Cost = 10 × FP + 500 × FN

7.1 Threshold moving (why and how)

[PLACEHOLDER: Moving threshold tradeoff + ROC figure]
[PLACEHOLDER: Overlapping distributions + ROC example]

Most classifiers output a probability of failure. Turning that probability into a hard decision requires choosing a threshold. The default threshold of 0.5 is arbitrary and usually not optimal for imbalanced, cost-sensitive problems.

Instead, we:

- scan thresholds (for example, from 0.01 to 0.99)
- compute validation false positives, false negatives, and Cost(t) at each threshold
- pick the threshold t* that minimizes cost on the validation set
- evaluate once on the test set using that chosen threshold

7.2 [placeholder: metric definitions + formulas (Precision/Recall/F1)]

We treat failure as the positive class. For any decision threshold, predictions fall into the standard confusion-matrix counts:

© 2026 Rula Awad. All rights reserved.

- **TP (True Positive):** predicted failure, actually failure
- **FP (False Positive):** predicted failure, actually no failure
- **FN (False Negative):** predicted no failure, actually failure
- **TN (True Negative):** predicted no failure, actually no failure

Precision: the fraction of relevant instances from among retrieved instances:

\text{Precision= frac{TP}{TP+FP}

Intuition: of all trucks you flagged for inspection, what fraction actually had a failure?

➔ Higher precision → fewer unnecessary checks (lower FP)

Recall / Sensitivity / TPR (how many true failures you catch)

\text{Recall}=\frac{TP}{TP+FN}

Intuition: of all true failures, what fraction did you detect?

➔ Higher recall → fewer missed failures (lower FN)

F1-score (balance score of precision + recall)

l\text{F1}=\frac{2 \cdot \text{Precision}\cdot \text{Recall}}{\text{Precision}+\text{Recall}}

Also:
\text{F1}=\frac{2TP}{2TP+FP+FN}

Precision and recall trade off as the threshold moves. Because missed failures are expensive in our setting, we don't rely on F1 alone, we report it, but we select the threshold based on cost.

[FIGURE PLACEHOLDER: Confusion matrix diagram (TP/FP/FN/TN)]

7.3 Imbalance Ratio (IR)

Imbalance Ratio (IR) here is a **dataset-only** number that tells you how *balanced* your class labels are.

➔ **IR = 1** → perfectly balanced (classes equally likely)
➔ **IR ≈ 0** → highly imbalanced (extreme class dominates)

To calculate the IR we use *normalised entropy*. Entropy measures uncertainty, and when classes are balanced you have high entropy (you can't predict with certainty which class you will get). When one class dominates, entropy is lower.

For our binary task (failure vs no-failure): p = failure rate and 1−p = no-failure rate

© 2026 Rula Awad. All rights reserved.

Entropy of the label distribution: $H(p)=-(p\log p+(1-p)\log(1-p))$

Normalize it to get a 0–1 imbalance indicator:

$IR=H(p)/\log 2$

IR is a **standalone heuristic** for *how severe is the class imbalance?* In our APS dataset, the failure rate is **< 2%** ($p<0.02$), so we expect IR to be much closer to **0** than **1** (for example, if $p=0.02$, $IR \approx 0.14$). This does **not** tell us anything about feature quality or how separable failures are. It just indicates that the label distribution is heavily skewed, which is why accuracy alone can be misleading and why we use *augmentation* and *cost-sensitive evaluation* in this exercise.

7.4 [placeholder: Evaluation Results]

8. Notebook 0X: LLMs + Agentic RAG (grounded answers)

In this notebook, we show how to build an LLM-based assistant and ground its answers in external content rather than relying on the model's internal memory alone. In Intelligent Transportation Systems, grounding is particularly useful because many questions must be answered against specific, authoritative documents (e.g. internal policies, safety bulletins, incident reports, maintenance logs, regulatory guidance). Concretely, an assistant can use grounded generation to: summarize safety bulletins and trend reports, answer questions about operating policies and procedures, and explain model outputs by attaching retrieved evidence (when available) so that decisions can be audited.

We use Retrieval-Augmented Generation (RAG) because it helps address common limitations of vanilla LLM usage. A standalone LLM may (i) hallucinate details, (ii) give answers that are plausible but not supported by the organization's documents, and (iii) lack up-to-date or organization-specific information. RAG mitigates this by ensuring the model's response is conditioned on the *right* source material at inference time. It also improves trust: users can see the text passages that the answer is based on, making responses easier to validate in safety-critical contexts.

8.1 Transformer Architecture & how LLMs work primer

[A more technical discussion for a more technical audience]

An LLM is a neural network trained on massive text corpora to model language as a probability distribution over tokens. LLMs for text are autoregressive and generate text one token at a time by predicting the next token given the previous tokens.

© 2026 Rula Awad. All rights reserved.

$$P(x_{1:T}) = \prod_{t=1}^{T} P(x_t \mid x_{<t})$$

The model learns parameters that minimize cross-entropy (next-token prediction loss) over many sequences. At inference time, it predicts a distribution for the next token, samples/selects a token, appends it to the prompt, and repeats.

LLMs tend to improve as you scale size, training data, size, compute as larger models have more capacity to capture the complexity and abstractions of language. They show better generalisation, reasoning, and instruction. However, performance relies on training quality, and alignment (fine-tuning, RLHF, instruction tuning). .).

The key point is even the best LLMs can still hallucinate, and we must put guardrails in place such as RAG to reduce this.

The Transformer Building Blocks

Token embeddings and positional information. Tokens are mapped into vectors (embeddings). Positional encodings is added so the model can represent order (since attention alone is permutation-invariant).

Multi-Head Attention (self-attention) Self-attention lets each token *look at* other tokens and compute a context-aware representation. And Multi-head means the model runs several attention operations in parallel, allowing it to capture different relationships (syntax, coreference, long-range dependencies, etc.) at once.

$$\mathrm{Attention}(Q,K,V)=\mathrm{softmax}\!\left(\frac{QK^{\top}}{d_k}\right)V$$

Q = queries, K = keys, V = values

The softmax term gives attention weights (which tokens matter to which token).

Feed-Forward Network (FFN). After attention, each position goes through a small neural net (applied independently per token position)

$$\mathrm{FFN}(h)=W_2\,\sigma\!\left(W_1 h+b_1\right)+b_2$$

Add & Norm (Residual connections & LayerNorm)
Each sub-layer is wrapped with (1) a residual connection (“skip connection”) to help gradient flow, and Layer Normalization for training stability.

$$h \leftarrow \mathrm{LayerNorm}\!\bigl(h+\mathrm{SubLayer}(h)\bigr)$$

Encoder Decoder & masked attention

© 2026 Rula Awad. All rights reserved.

The original transformers were encoder-decoder models. Encoders read the input sequence and produce contextual representations. Decoder generates the output sequence token by token.

Encoder Layer
Mutli head self attention (unmasked: can attend to all input tokens), the feedforward network, and residual connections and layer norms.

Decoder Layer
1. Masked multi-head self-attention Masking prevents the model from looking at f*uture* tokens during generation, enforcing autoregressive behaviour
2. Cross attention over encoder outputs (in encoder-decoder models). The decoder attends to the encoder's output to condition generation on the input sequence
3. The FFN
4. Add & Norm after each sublayer

Most LLMs you use locally are decoder-only Transformers. They rely on masked self-attention + FFN + Add&Norm, stacked many times without a separate encoder module.

For RAG, conditioning on documents is done by inserting retrieved text into the prompt, not via encoder–decoder cross-attention.

LLM Positioning: open vs closed models

Open weight models: means you can download and see the model weights, run inference locally on your infrastructure, fine-tune or adapt the model. This gives control over privacy, latency, and governance.

Closed models: access through APIs, no control over weights. Out of the box means you tradeoff transparency, flexibility in deployment and (sometimes) data control.

We use open source models so we can have local inference and customisation, although, closed models are more convenient.

Modality: text only vs multimodal

Text only: inputs/ outputs is text
Multimodal: can accept other modalities such as images, audio, video which is useful in our case study and ITS as tasks involve camera feeds, signs, scenes, movement, diagrams etc.

[FIGURE PLACEHOLDER: Transformer Diagram]

8.2 RAG

© 2026 Rula Awad. All rights reserved.

RAG stands for Retrieval-Augmented Generation. Instead of generating an answer from the language model alone, we first convert the user query into an embedding vector, retrieve the most relevant text chunks from a vector database (or search index), and pass those chunks to the LLM as additional context. We then instruct the LLM to answer using only the retrieved context (or to clearly state when the context is insufficient). This is often described as giving the LLM “open-book access” to a curated set of documents: the model is still doing natural-language reasoning and synthesis, but it is anchored to specific, retrievable evidence.

This grounding reduces hallucinations, keeps answers aligned with the current document set, and enables citation-style evidence for safety- and policy-critical questions.

Context aware insights

- [results from notebook]
- [diagrams]

What you control in the RAG demo

- Document source: change the URL (or load PDFs, manuals, policies).
- Chunking strategy: chunk_size and chunk_overlap determine retrieval granularity.
- Embedding model: affects what 'similarity' means.
- Retriever top-k: more chunks can help recall but may dilute focus.
- Prompt rules: instruct the model to say 'I don't know' when context is insufficient.

8.2 Agents

9. [placeholder: Key Takeaways]

- Single metrics (accuracy/F1) misleading alone
- *set* of KPIs aligned to operational objectives (cost + FP/FN + recall/precision)
- Real-world datasets require domain context (how data is collected, what failures mean).
- No one-size-fits-all: method effectiveness depends on intrinsic dataset characteristics and complexity.

Getting Started with notebooks / github:

© 2026 Rula Awad. All rights reserved.

Acronyms

- ➔ **ITS** — Intelligent Transportation Systems
- ➔ **IEEE** — Institute of Electrical and Electronics Engineers
- ➔ **ADAS** — Advanced Driver Assistance Systems
- ➔ **APS** — Air Pressure System
- ➔ **PCA** — Principal Component Analysis
- ➔ **DDPM** — Denoising Diffusion Probabilistic Model
- ➔ **RAG** — Retrieval-Augmented Generation
- ➔ **LLM** — Large Language Model
- ➔ **SMOTE** — Synthetic Minority Over-sampling Technique
- ➔ **KS test** — Kolmogorov–Smirnov test
- ➔ **TP/FP/TN/FN** — Confusion matrix counts
- ➔ **ROC / PR** — Receiver Operating Characteristic / Precision–Recall
- ➔ **AUC** — Area Under the Curve
- ➔ **CFP/CFN** — Cost of False Positive / Cost of False Negative
- ➔ **TPR/FPR** — True Positive Rate / False Positive Rate

Appendix: troubleshooting checklist

- If your raw APS CSV load fails or columns look wrong: verify you used skiprows=20.
- If your missingness plots show 0 missing values: confirm 'na' strings were converted to NaN.
- If diffusion training diverges: reduce learning rate, increase batch size, or train in PCA space only.
- If RAG returns irrelevant context: reduce chunk size, increase overlap, or change embedding model.
- If RAG hallucinates: strengthen the prompt instruction to only use context and return 'I don't know' otherwise.

References

- APS Failure at Scania Trucks (IDA 2016 dataset).
- Ho et al., *"Denoising Diffusion Probabilistic Models"* (DDPM), 2020.
- Transformer architecture (Vaswani et al., 2017) and modern open-source LLM tooling (Ollama, LangChain, Chroma).

© 2026 Rula Awad. All rights reserved.